探索（search）

1．目的

「線形探索」、「番兵」、「2分探索」など、様々な探索アルゴリズムを学ぶと同時時に、ポインタの使用方法の定着を図ります。

2．技術的知識

（1）strcmp()関数

C言語の標準関数には、文字列の一致を確認するstrcmp()関数があります。

```
int strcmp(char *s1, char *s2);
```

※s1，s2は、文字列の入っている配列名や"hamako"などの「"」で囲まれた文字列です。すなわち、文字列へのポインタになります。

この関数は、s1が指す文字列とs2が指す文字列が一致すれば「0」を返します。また、戻り値の正負によって、その2つの文字列の大小（順序）を確認することができます。ただし、このときの順序は、アルファベット順（辞書順）ではなく文字コード順になります。

［例］

	戻り値	
strcmp("hamako", "hamako")	0	一致
strcmp("technical", "hamamamatsu")	正	"technical" ＞ "hamamamatsu"
strcmp("electrical", "information")	負	"electrical" ＜ "information"

（2）strcpy()関数

C言語の標準関数のには、文字列をコピーするstrcpy()関数があります。

```
char *strcpy(char *s1, char *s2);
```

※s2が指す文字列を、s1が指す文字配列にコピーします。コピー元とコピー先が重なる場合の動作は未定です。戻り値はs1の値（コピー先の文字列を指すポインタ）を返します。

［例］

```
char str1[50];
char str2[50];
    …
strcpy( str1, "hamamatsu");
```

文字配列str[]に文字列"hamamatsu"がコピーされます。**但し、strcpy()は、コピー先領域の大きさやコピー元の文字数などはチェックしないので、コピー先の配列には、コピー元の文字列が格納できる十分な大きさの領域を確保しておかなければなりません。**

そのため、多くの場合 strcpy() を使う場面では、コピーする文字数を引数で指定する strncpy() の使用が推奨されています（strncpy()については、各自で調べて下さい）。

なお、strcmp()およびstrcpy()関数を利用するときは、ヘッダファイルstring.hを必ずインクルードしておく必要があります。

（3）線形探索法（逐次探索法）

線形探索は、もっとも単純な探索方法で、目的のデータを、検索対象となるデータと先頭から順に比較する操作を繰り返す方法です。目的のデータが見つかるか、すべてのデータを比較し尽くしたときに繰り返しを終了します。

［例］"JasonSpice"の探索（目的のデータが存在する場合）

1	LeeStensness	←比較→	JasonSpice	（不一致）	
2	CharlesRiechelmann	←比較→	JasonSpice	（不一致）	
3	PaulMitchell	←比較→	JasonSpice	（不一致）	
4	EroniClarke	←比較→	JasonSpice	（不一致）	
5	JasonSpice	←比較→	JasonSpice	（ 一致 ）	… **発見**
6	XavierRush				
7	CraigInnes				
8	LeoLafaialii				
9	ApisaiNaevo				
10	MichaelJones				

［例］"JoeliVidiri"の探索（目的のデータが存在しない場合）

1	LeeStensness	←比較→	JoeliVidiri	（不一致）	
2	CharlesRiechelmann	←比較→	JoeliVidiri	（不一致）	
3	PaulMitchell	←比較→	JoeliVidiri	（不一致）	
4	EroniClarke	←比較→	JoeliVidiri	（不一致）	
5	JasonSpice	←比較→	JoeliVidiri	（不一致）	
6	XavierRush	←比較→	JoeliVidiri	（不一致）	
7	CraigInnes	←比較→	JoeliVidiri	（不一致）	
8	LeoLafaialii	←比較→	JoeliVidiri	（不一致）	
9	ApisaiNaevo	←比較→	JoeliVidiri	（不一致）	
10	MichaelJones	←比較→	JoeliVidiri	（不一致）	… **存在せず**

（4）番兵（sentinel：センチネル）

（3）の線形探索では、目的のデータが存在する場合は、見つかった時点で繰り返しを終了し、存在しない場合はすべてのデータを比較し終わったとき、繰り返しが終了となります。このため、繰り返しの終了条件は、例えば、

```
while（目的のデータが見つからない  &&  まだデータが残っている）{
   …
}
```

のように、複雑になってしまいます。また、この終了条件は論理的には正しいように見えますが、実際には、**C言語でこの条件をそのまま実装しても、おそらく正しく動作しません（なぜでしょう？）**。

ところが、検索対象データの最後尾に目的のデータを格納しておくことにより、この終了条件をずっと単純にでき、また、確実に動作させることができます。この最後尾に格納するデータのことを番兵(sentinel)といいます。もちろん、検索対象データが格納されている領域に、さらに1つ分データを格納する余裕がある必要があります。

［例］"JoeliVidiri"の探索（目的のデータは存在しませんが、必ず番兵とは一致します）

```
 1  LeeStensness          ←比較→   JoeliVidiri   （不一致）
 2  CharlesRiechelmann    ←比較→   JoeliVidiri   （不一致）
 3  PaulMitchell          ←比較→   JoeliVidiri   （不一致）
 4  EroniClarke           ←比較→   JoeliVidiri   （不一致）
 5  JasonSpice            ←比較→   JoeliVidiri   （不一致）
 6  XavierRush            ←比較→   JoeliVidiri   （不一致）
 7  CraigInnes            ←比較→   JoeliVidiri   （不一致）
 8  LeoLafaialii          ←比較→   JoeliVidiri   （不一致）
 9  ApisaiNaevo           ←比較→   JoeliVidiri   （不一致）
10  MichaelJones          ←比較→   JoeliVidiri   （不一致）
11  JoeliVidiri           ←比較→   JoeliVidiri   （ 一致 ）
    ↑
    └─ 最後尾に目的のデータと同じものを格納しておく
```

この場合、必ず目的のデータは見つかることになるので、繰り返しの終了条件は次のように非常に単純になります。

```
while（目的のデータが見つからない）{
  …
}
```

なお、目的のデータが存在した場合としなかった場合との区別は、目的のデータが番兵の位置（すなわちデータの最後尾）で見つかったかどうかで判断できます。

（6）2分探索法

線形探索では、検索対象データの最初の方に一致するデータがあれば短時間で見つかりますが、最後の方に一致するデータがある場合やデータが存在しない場合には処理に大変時間がかかってしまいます。検索対象データが、規則的に並んでいる場合（昇順や降順）は、2分探索という方法を用いることにより、処理時間を大幅に短縮することができます。

例えば、検索対象データが次のようにアルファベットの昇順に並んでいるような場合、2分探索で"JonahLomu"を検索する手順は以下のようになります。

① data[i]

			②③	④⑤	⑥以降
low	0	AdrianCashmore			
	1	AndrewBlowers			
	2	AndrewRoose			
	3	ApisaiNaevo			
	4	BrianLima			
	5	CalebRalph			
	6	CarlosSpencer			
	7	CharlesRiechelmann			
	8	CraigDowd			
	9	CraigInnes			
	10	DylanMika			
	11	EroniClarke			
mid	12	JasonSpice			
	13	JoeliVidiri	←low	←low	←low
目標→	14	JonahLomu			←high
	15	LeeLidgard		←mid	
	16	LeeStensness			
	17	LeoLafaialii			
	18	MarkCarter		←high	
	19	MichaelJones	←mid		
	20	Ofisa"Junior"Tonu'u			
	21	OloBrown			
	22	PaulMitchell			
	23	RobinBrooke			
	24	SeanFitzpatrick			
high	25	XavierRush	←high		

① まず、検索対象の先頭をlow、最後尾をhighとする。low～highの範囲で、中央のデータ(mid = (low + high) / 2 = (0 + 25) / 2 = 12)と目的のデータ"JonahLomu"とが一致するかを、strcmp(target, data[mid])などで確認します。

② ここでは、一致せずstrcmp()関数の値は「正」("JonahLomu" > "JasonSpice")なので、目的の"JonahLomu"は、mid + 1(=13)とhigh(=25)の間にあるはずです。

③ そこで、mid + 1をlowとし、新たにmid(=(low + high) / 2 = (13 + 25) / 2 = 19)を計算し、同様にtargetとdata[mid]が一致するか確認します。

④ strcmp()関数の値は「負」("JonahLomu" < "MichaelJones")となるので、目的の"JonahLomu"は、low(=13)とmid - 1(=18)の間にあるはずです。

⑤ mid - 1をhighとし、③と同様にmidを計算し(=15)し、targetとdata[mid]が一致するか確認します。

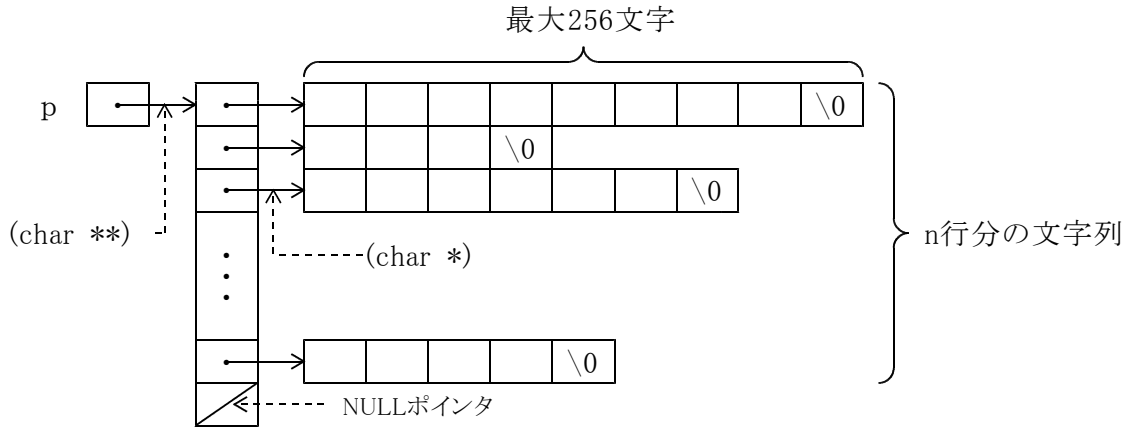
⑥ 以下同様に、この作業を繰り返し、targetとdata[mid]が一致すればmidの位置が目的のデータの位置になります。また、low > highとなった場合は、一致するデータなかったことになります。

３．課題A

以下の(1)～(4)のプログラムを作成しなさい。なお、**プログラム終了時には、動的に確保した領域はすべて正しく解放すること**。

(1)　"/home/pub/practice/words.dat"というファイルには1行に1つずつ文字列が格納されており、ファイルの先頭行には格納されている文字列の行数が記述してあります。

このファイルを**標準入力より読み込み**、下図のように、各行の文字列をすべて動的に確保した領域に格納する関数 allocWords() と、allocWords()で確保した領域を解放する関数 freeWords() を作りなさい。なお、1行の最大文字数は、空文字も含め256文字であることが保証されているものとします。



ここで、allocWords() と freeWords() の外部仕様は次のようにします。

```
char **allocWords(int *line_nums);
```

[引　数] 入力した文字列の行数を格納する変数へのポインタ

[戻り値] 入力行を正常に確保できた　：　全入力行を格納した配列へのポインタ
入力行の領域確保に失敗した ：　NULL

[機　能] 標準入力より文字列を行単位で入力し、上図の構造を動的に作る。すべての文字列が正常に入力できたとき、*line_nums には入力した文字列の行数が入る。入力行の領域確保に失敗したときは、*line_nums は 0 になる。

```
void freeWords(char **lines, int line_nums);
```

[引　数] **lines**　：　allocWords()で入力した全文字列を格納している配列へのポインタ
line_nums：　linesに格納されている文字列の行数

[戻り値] なし

[機　能] allocWords()で確保した領域をすべて解放する

これら関数の本体は"allocwords.c"に、プロトタイプ宣言は"allocwords.h"というヘッダファイルに記述し、以下の設問で利用しなさい。なお、各設問のプログラムは、おおよそ次のような形になります。

(添付ファイル：allocwords.c allocwords.h)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "allocwords.h"

int main(int argc, char *argv[])
{
  int n;
  char **p;

    …  その他の変数宣言やコマンドライン引数の処理など  …

  if ((p = allocWords(&n)) == NULL) {
    fprintf(stderr, "words allocation failed.\n");
    exit(8);
  }
```

p[] の各文字列に対して探索

```
  freeWords(p, n);

  return(0);
}
```

(2)　(1)で作成したallocWords()関数で、"/home/pub/practice/words.dat"を読み込み、その文字列の中から、コマンドラインに指定した文字列を(番兵を用いない)線形探索法により検索するプログラムを作成しなさい。文字列が見つかった場合はその行位置を表示し、見つからなかった場合は"not found"と表示しなさい。

なお、文字列の行位置は、先頭行を1行目と数えるものとします。

(添付ファイル：search2.c)

－実行例－

```
$ ./a.out httpd < /home/pub/practice/words.dat
line 391: 'httpd' found.
$ ./a.out samba < /home/pub/practice/words.dat
'samba' dose not found.
```

(3)　(2)のプログラムを番兵(sentinel)を用いたものに書き換えなさい。

(添付ファイル：search3.c)

(4)　(2)と同様のプログラムを2分探索法により作成しなさい。

なお、検索対象のデータには、"/home/pub/practice/words.sorted"を使用しなさい。このファイルは、(1)(2)で用いた"/home/pub/practice/words.dat"を、文字コードの昇順(小さい順)でソートしたものです。

(添付ファイル：search4.c)

4．課題B

(1) 検索対象のデータ数がn個であるとする。このとき、線形探索法、2分探索法それぞれについて、目的のデータが見つかるまでの最大比較回数を示しなさい。

(2) ハッシュ法について調べなさい。また、余力のある者は**[課題A(2)]**と同様の検索プログラムを、ハッシュ法を用いて作りなさい。